

TAREA Sistema RAG con Transcripciones de Khan Academy

Descripción General

El objetivo de este proyecto es construir un sistema de **Retrieval-Augmented Generation (RAG)** que permita a los usuarios interactuar en lenguaje natural con el contenido educativo de los cursos de **Khan Academy**, a través de sus transcripciones en texto.

El sistema deberá integrar todos los conceptos trabajados durante el curso: carga y limpieza de datos, generación de embeddings, base de datos vectorial, recuperación semántica con criterios de selección, memoria de conversación y una interfaz de usuario con Gradio.

Fuente de Datos

Las transcripciones de los vídeos de Khan Academy están disponibles como un dataset público en Hugging Face. El dataset contiene registros en formato **JSON** con campos que incluyen el identificador del vídeo, el título, el tema y la transcripción del contenido.

- **Dataset:** [iblai/ib1-khanacademy-transcripts](#)
- **Formato:** JSON con campos como title, content, language, url, etc.

Importante: No es necesario utilizar todos los registros del dataset. Se recomienda seleccionar un subconjunto representativo de unos cuantos vídeos para mantener tiempos de procesamiento razonables durante el desarrollo.

Fases del Proyecto

Fase 1 — Descarga y Limpieza del Dataset

El JSON original del dataset puede contener:

- Campos irrelevantes o vacíos
- Transcripciones con ruido ([música], [aplausos], tiempos de subtítulo, caracteres especiales)
- Registros duplicados o con transcripciones demasiado cortas para ser útiles

Se deberá realizar una **limpieza y normalización** del JSON antes de indexarlo en la base de datos vectorial. El resultado de esta limpieza debe guardarse en un archivo limpio (p. ej. train_clean.json) que será la entrada al pipeline de indexación.

Tareas mínimas de limpieza:

- Eliminar o filtrar registros con transcripciones vacías o demasiado cortas (< 50 palabras)
- Eliminar etiquetas de subtítulos y marcadores de tiempo
- Normalizar espacios en blanco y saltos de línea
- Seleccionar y conservar únicamente los campos relevantes para el RAG (title, content, url)
- Opcionalmente: dividir las transcripciones largas en fragmentos (chunks) coherentes

Fase 2 — Indexación en Base de Datos Vectorial

Una vez limpio el dataset, se generarán **embeddings** de cada fragmento de transcripción y se almacenarán en una base de datos vectorial.

Opciones de base de datos vectorial:

Opción	Descripción
FAISS <i>(recomendada)</i>	En memoria, sin dependencias externas, ideal para desarrollo
ChromaDB in-memory <i>(recomendada)</i>	En memoria con API de alto nivel, fácil de usar con LangChain
ChromaDB con contenedor Docker	Persistente, requiere levantar un servicio externo
Qdrant con Docker	Alternativa robusta con soporte para filtros avanzados

Se recomienda utilizar una base de datos vectorial **en memoria** (FAISS o ChromaDB) para simplificar el despliegue. Si se desea persistencia entre sesiones, se puede usar ChromaDB o Qdrant con un contenedor Docker.

Modelo de embeddings sugerido: nomic-embed-text (disponible en Ollama)

TAREA Sistema RAG con Transcripciones de Khan Academy

Fase 3 — Pipeline RAG

Se implementará un pipeline RAG completo usando **LangChain** y **Ollama** como backend de inferencia local.

El pipeline deberá incluir:

- Retriever** — Recuperación semántica de los fragmentos más relevantes desde la base de datos vectorial según la consulta del usuario
- Criterios de selección** — El retriever deberá configurarse con al menos los siguientes parámetros:
 - Número de fragmentos a recuperar (k)
 - Umbral mínimo de similitud (score_threshold)
 - Opcionalmente: filtrado por metadatos (p. ej. por topic)
- Memoria de conversación** — El sistema deberá mantener el historial de la conversación para poder responder preguntas de seguimiento y referencias contextuales
- Prompt engineering** — El prompt del sistema deberá indicar claramente al LLM que base sus respuestas únicamente en el contexto recuperado, e indicar cuando la respuesta no esté disponible en el material

Modelo LLM sugerido: llama3.2 o mistral para generación de respuestas. Para los embeddings, se recomienda nomic-embed-text (modelos disponibles en Ollama)

Fase 4 — Interfaz Gradio

Se desarrollará una **interfaz de usuario con Gradio** que permita:

- Un campo de texto para introducir preguntas en lenguaje natural
- Un historial de conversación visible (tipo chatbot)
- Un panel lateral o desplegable que muestre los **fragmentos recuperados** (fuentes usadas para generar la respuesta)
- Controles para ajustar los parámetros del retriever:
 - Número de fragmentos k (slider, rango 1-10)
 - Umbral de similitud (slider, rango 0.0-1.0)
 - Filtro por tema/área temática (dropdown)
- Un botón para **limpiar el historial** de conversación
- Se pueden incluir funcionalidades adicionales como exportar la conversación o mostrar enlaces a los vídeos originales

Criterios de Evaluación

Criterio	Peso	Descripción
Limpieza del dataset	15%	El JSON se ha limpiado correctamente: se eliminan ruidos, campos irrelevantes y registros no aptos. El proceso está documentado y es reproducible.
Calidad del RAG	30%	Las respuestas generadas son relevantes, están fundamentadas en el contexto recuperado y el sistema indica correctamente cuando no dispone de información.
Criterios de selección del retriever	15%	Se han implementado y expuesto al usuario al menos dos parámetros configurables (k, umbral de similitud, filtro por topic). Se demuestra su efecto en los resultados.
Memoria de conversación	10%	El sistema recuerda el contexto de turnos anteriores y responde correctamente a preguntas de seguimiento o referencias anafóricas.
Interfaz Gradio	15%	La interfaz es funcional, usable e incluye como mínimo: chat, visualización de fuentes recuperadas y controles de parámetros.
Documentación y código	15%	El código está organizado, comentado en los puntos clave y se incluye un README.md con instrucciones para ejecutar el proyecto.

Entregables

- Script de limpieza** (clean_json.py): limpia el JSON original y produce train_clean.json
- Script de indexación** (index_data.py): genera los embeddings e indexa el dataset en la base de datos vectorial
- Aplicación principal** (app.py): pipeline RAG + interfaz Gradio completamente funcional
- dependencies** (requirements.txt): lista de dependencias necesarias para ejecutar el proyecto

TAREA Sistema RAG con Transcripciones de Khan Academy

5. **Dataset limpio** (`train_clean.json`): resultado de la limpieza (puede ser un subconjunto)
6. **README.md**: instrucciones de instalación, configuración de modelos y ejecución

Restricciones Técnicas

- El LLM y el modelo de embeddings deben ejecutarse **localmente con Ollama** en el entregable final. en el desarrollo se pueden usar servicios de inferencia en la nube, pero el resultado final debe ser completamente local.
- El proyecto debe ejecutarse en el entorno virtual del curso (`.venv`)
- Se debe usar **LangChain** como framework principal de orquestación

Recursos

- LangChain RAG docs: <https://python.langchain.com/docs/tutorials/rag/>
- Dataset Khan Academy: <https://huggingface.co/datasets/iblai/ibla-khanacademy-transcripts>
- HuggingFace Datasets: <https://huggingface.co/docs/datasets/>
- Gradio docs: <https://www.gradio.app/docs/>
- ChromaDB: <https://docs.trychroma.com/>
- FAISS: <https://faiss.ai/>